

Git Essentials

14/05 y 15/05

Dra. Ing. Paola B. Torres y Esp. Ing. Daniela Armijo



Esp. Ing. Daniela Armijo

- Ing. Química
- Especialista en Docencia Univ. de UNCuyo
- Docente en: Análisis Matemático I, Álgebra y Geometría Analítica, Física I, y Análisis Matemático II
- Estudiante en Diplomatura de Ciencia de Datos en UNC.

Dra. Ing. Paola B. Torres

- Dra. en Ingeniería.
- Posdoctoral en Aprendizaje Automático
- Grupo Vinculado Bionanotecnología y sistemas complejos UTN-CONICET-INFAP
- Ayudante en Análisis Matemático y Álgebra y Geometría Analítica.

1. Introducción

- ¿Qué es Git?
- ¿Cómo funciona Git?
- ¿Cómo se relaciona Git con GitHub u otros servicios?

2. Comandos básicos

- Comandos útiles de terminal (bash)
- Configuración básica de Git
- Creación de un repositorio Git
- Confirmar cambios en el repositorio
- Gestión de archivos
- Historial de cambios y estado del repositorio
- Hash: identificador de commits

3. Colaboración

- Sincronización
- Ramas
- Resolución de conflictos
- GitHub funcionalidades
- Buenas practicas

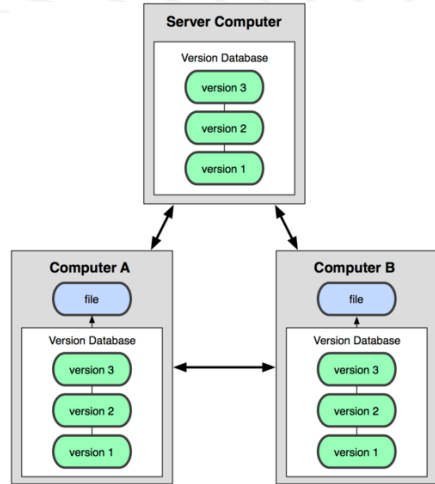
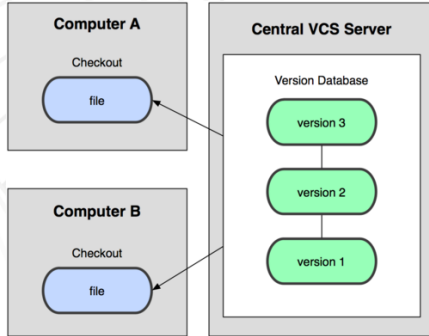
4. Referencias

¿Qué es Git?

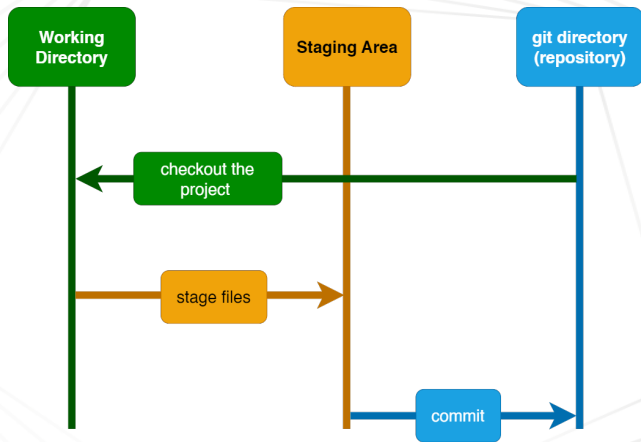


- Fue creado por Linus Torvalds el creador del kernel de Linux.
- Es Open Source.
- Es una herramienta de control de versiones distribuido
- Permite trabajar en un proyecto sin necesidad de estar conectados a un servidor central.
- Un sistema de control de versiones es una herramienta que se utiliza para mantener un registro de los cambios que se hacen en un proyecto
- Documentación: [Web oficial git](#)

Centralizado vs Distribuido



¿Cómo funciona Git?



Plataformas



¿Cómo se relaciona Git con GitHub?

- Es una plataforma de alojamiento de código en la nube que utiliza el **sistema de control de versiones Git**.
- Colaboración en proyectos privados y abiertos.
- Herramientas adicionales: seguimiento de problemas, integración continua, revisión de código y colaboración en proyectos de código abierto.



1. Introducción

- ¿Qué es Git?
- ¿Cómo funciona Git?
- ¿Cómo se relaciona Git con GitHub u otros servicios?

2. Comandos básicos

- Comandos útiles de terminal (bash)
- Configuración básica de Git
- Creación de un repositorio Git
- Confirmar cambios en el repositorio
- Gestión de archivos
- Historial de cambios y estado del repositorio
- Hash: identificador de commits

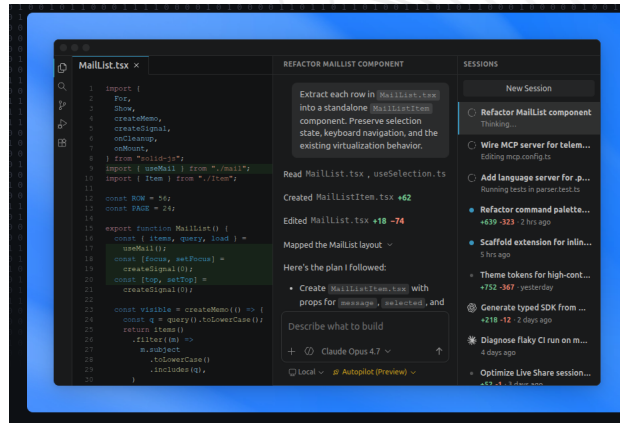
3. Colaboración

- Sincronización
- Ramas
- Resolución de conflictos
- GitHub funcionalidades
- Buenas practicas

4. Referencias

Editores de Código

- VS Code
- Notepad++
- VIM



Comandos utiles de terminal

```
$ ls  
$ cd <directorio>  
$ cd ..  
$ pwd  
$ clear  
$ mkdir <nombre_directorio>  
$ touch <nombre_archivo>
```

- ls (list): listar archivos
- cd (change directory): cambiar de carpeta
- pwd (print working directory): muestra directorio actual
- mkdir (make directory): crear crea una carpeta nueva

Comandos utiles de terminal

```
$ ls  
$ touch <nombre_archivo>  
$ rm <nombre_archivo>  
$ mv <nombre_archivo> <directorio>  
$ cp <nombre_archivo> .  
$ cp <nombre_archivo> <directorio>
```

- touch: crear archivos ejemplo
- rm (remove): eliminar archivos
- mv (move): mover archivos
- cp (copy): copiar archivo

Configuración básica de Git

```
git config  
git config --global user.name 'nombre'  
git config --global user.email 'email'
```

Ref: Documentación [git config](#)

Creación de un repositorio desde GitHub

Crear un repositorio en GitHub

Vamos a [GitHub](#)

Clonar un repositorio desde GitHub: HTTPS

```
git clone https://github.com/user/repositorio.git
```

Clonar un repositorio desde GitHub: SSH

```
git clone git@github.com:/user/repositorio.git
```

Ref: Documentación para [autenticación ssh](#)

Creación de un repositorio Git

```
mkdir <nombre_carpeta>  
git init
```

Conectar repositorio a la nube de GitHub

```
git remote add origin git@github.com:user/repositorio.git  
git branch -M main  
git push -u origin main
```

Confirmar cambios en el repositorio

Primero: Agregar archivos

```
$ git add [archivo]
```

Segundo: git commit

```
$ git commit -m 'insertar mensaje simple y preciso'
```

Tercero: git push

```
$ git push
```


Gestión de archivos

Agregar archivos

```
$ git add <nombre_archivo>
```

Mover archivos

```
$ git mv <nombre_archivo>
```

Eliminar archivos

```
$ git rm <nombre_archivo>
```

Historial de cambios y estado del repositorio

Estado de los archivos

```
$ git status
```

Historial

```
$ git log
```

```
$ git log --pretty=oneline
```

```
$ git log --graph
```

Identificar commits

```
* | commit 98442e283a2826021bea6e15e58204423ddb1c9b
  | Author: Paola <paotorres89@gmail.com>
  | Date:   Fri Sep 19 11:17:24 2025 -0300
  |
  |     added reverse scheduled and ss
  |
* | commit f826b302aa0ba1a32c03b345e0b79024240a3c82 (main)
  | Author: hernandezhdd <71747228+hernandezhdd@users.noreply.github.com>
  | Date:   Wed Jul 16 13:45:27 2025 -0300
  |
  |     termino de comentar convLSTM-torch.ipynb
```

¿Qué es un Hash?

Una función criptográfica hash es un algoritmo matemático que transforma cualquier bloque arbitrario de datos en una nueva serie de caracteres con una longitud fija (40 caracteres)

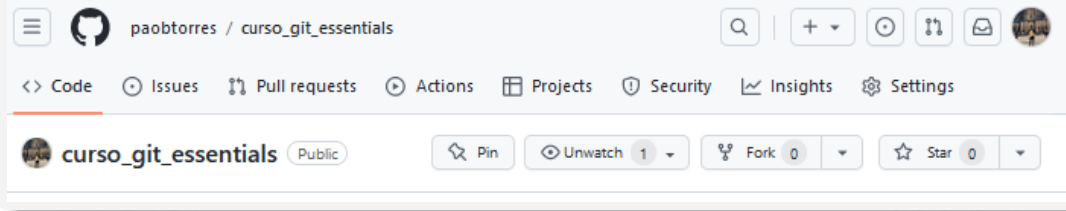
```
* | commit 98442e283a2826021bea6e15e58204423ddb1c9b
  | Author: Paola <paotorres89@gmail.com>
  | Date:   Fri Sep 19 11:17:24 2025 -0300
  |
  |     added reverse scheduled and ss
  |
* | commit f826b302aa0ba1a32c03b345e0b79024240a3c82 (main)
  | Author: hernandezhdd <71747228+hernandezhdd@users.noreply.github.com>
  | Date:   Wed Jul 16 13:45:27 2025 -0300
  |
  |     termino de comentar convLSTM-torch.ipynb
```

¿Para que nos sirve el Hash?

```
$ git diff <hash_a> <hash_b>
$ git diff --name-only <hash_a> <hash_b>
$ git checkout <hash_commit>
$ git reset -- hard <hash>
```

Fork (Bifurcación)

Permite copiar un repositorio en nuestro GitHub para poder realizar cambios en un repositorio publico para el cual no tenemos permisos.



Pull request

The screenshot shows a GitHub pull request for the 'hello-git' repository, which is a fork of 'paolabeatriz/hello-git'. The interface is in dark mode. At the top, it shows the repository name 'hello-git' with a 'Public' label, and buttons for 'Pin' and 'Watch'. Below this, it indicates the fork source 'forked from paolabeatriz/hello-git'. The main section shows the current branch 'main' with a dropdown, '1 branch', and '0 tags'. There are buttons for 'Go to file', 'Add file', and 'Code'. A status bar indicates 'This branch is 1 commit ahead, 1 commit behind paolabeatriz:main.' with 'Contribute' and 'Sync fork' options. The commit history shows three commits: 'Initial commit' for 'README.md' (1 hour ago), 'script to find the max value' for 'new-script.py' (now), and 'Rename test-script-py to test-script.py' for 'test-script.py' (1 hour ago).

hello-git Public
forked from paolabeatriz/hello-git

main 1 branch 0 tags
Go to file Add file Code

This branch is 1 commit ahead, 1 commit behind paolabeatriz:main.
Contribute Sync fork

paobtorres script to find the max value 7df97c3 now 8 commits

README.md	Initial commit	1 hour ago
new-script.py	script to find the max value	now
test-script.py	Rename test-script-py to test-script.py	1 hour ago

Figura: Confirmando los commits locales antes de enviar.

Pull request

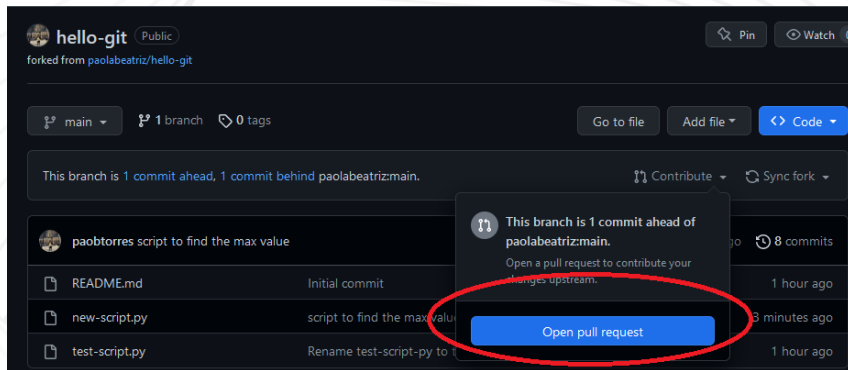


Figura: Abriendo la solicitud de Pull Request en el repositorio.

Pull request

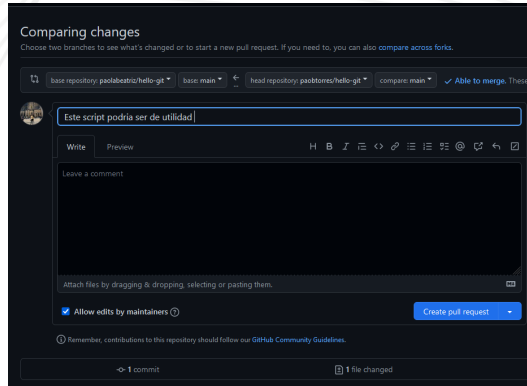


Figura: Completando la descripción y enviando la propuesta.

Pull request

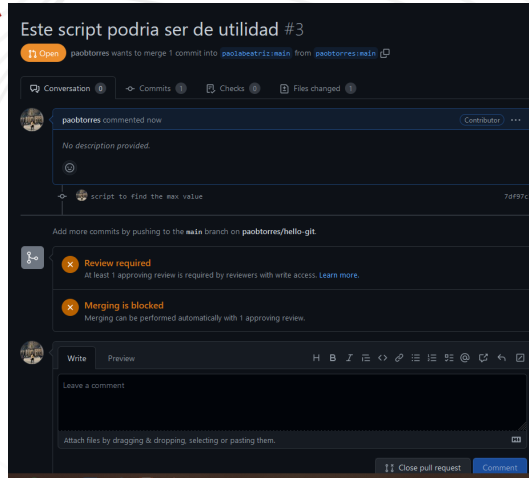
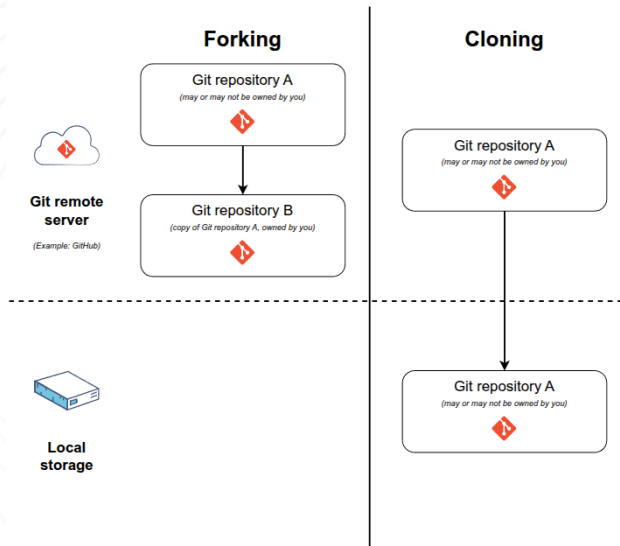


Figura: Pull Request creado correctamente y listo para revisión.



Práctica

Añadir un archivo en la carpeta `actividad_uno` que tenga como nombre su usuario de GitHub.

Para esto deberán:

- 1 Realizar un fork del repositorio del curso [gitesentials2026](#)
- 2 Debera clonar el repositorio copiando la direccion desde su github (`$ git clone ...`)
- 3 En la carpeta `actividad_uno` crear un archivo con su nombre de usuario de gitbub.
Ejemplo `paobtorres.txt`
- 4 Una vez creado el archivo recuerde hacer: `git add`, `git commit`, `git push`
- 5 Ultimo paso: desde github debera crear un Pull Request para que los cambios se reflejen en el repositorio del curso.

1. Introducción

- ¿Qué es Git?
- ¿Cómo funciona Git?
- ¿Cómo se relaciona Git con GitHub u otros servicios?

2. Comandos básicos

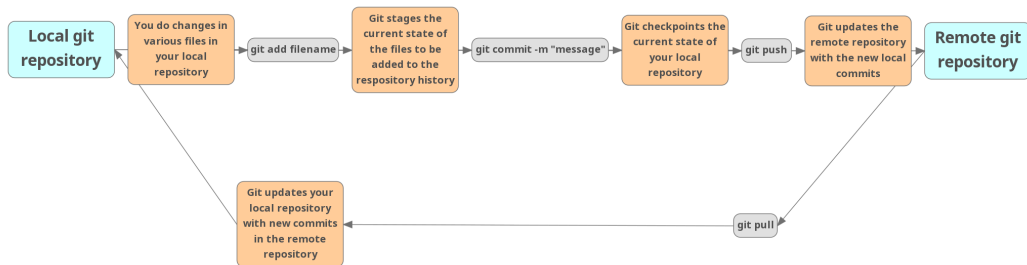
- Comandos útiles de terminal (bash)
- Configuración básica de Git
- Creación de un repositorio Git
- Confirmar cambios en el repositorio
- Gestión de archivos
- Historial de cambios y estado del repositorio
- Hash: identificador de commits

3. Colaboración

- Sincronización
- Ramas
- Resolución de conflictos
- GitHub funcionalidades
- Buenas practicas

4. Referencias

Workflow



Sincronización

Fetch

Se utiliza para descargar el historial de cambios del repositorio remoto al repositorio local, pero sin aplicar los cambios.

Pull

Descarga los cambios del repositorio remoto y los fusiona con los cambios locales. Si hay conflictos en la fusión git intentara de combinar los cambios por defecto.

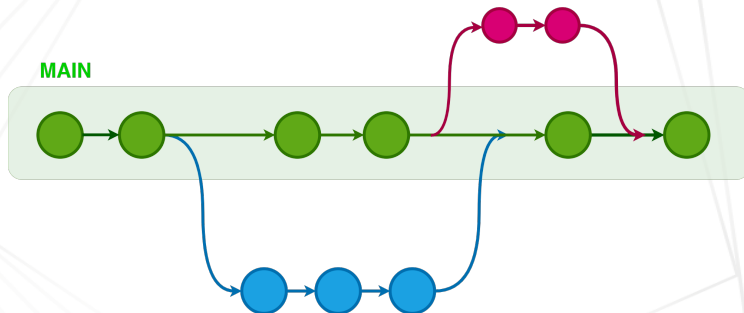
Comandos de Sincronización

```
$ git fetch  
$ git config pull.rebase false  
$ git pull
```

Ramas (git branch)

¿Para que sirven las ramas?

Permite trabajar de manera paralela sin realizar cambios en el código principal



Ejemplo en [github/pyMBE](https://github.com/pyMBE)

Gestión de ramas

```
$ git branch <nombre_rama> (Crear)
$ git branch <nombre_rama> -d (Eliminar)
$ git switch <nombre_rama> (Cambiar a una rama)
$ git checkout -b <nombre_rama> (Cambiar a una rama)
$ git merge bugfix_branch main (Integrar una rama)
```

git switch vs git checkout

git switch: Comando específico para cambiar entre distintas ramas

git checkout: tiene varias funciones, incluyendo la capacidad de cambiar entre ramas, hash, tags y commits

Documentación [git branch](#)

Creación de un repositorio Git

```
mkdir <nombre_carpeta>  
git init
```

Conectar repositorio a la nube de GitHub

```
git remote add origin git@github.com:user/repositorio.git  
git branch -M main  
git push -u origin main
```

Git push

```
git branch bugfix  
git switch bugfix  
git push
```

fatal: The current branch bugfix has no upstream branch.
To push the current branch and set the remote as upstream, use
`git push -set-upstream origin bugfix`

```
git push -set-upstream origin bugfix  
git push -u origin bugfix
```

Cambios temporales

Stash

Permite guardar temporalmente los cambios que hemos realizado en un archivo, o conjunto de archivos, sin tener que hacer commit

Reservar cambios

```
$ git stash  
$ git stash list  
$ git stash pop  
$ git stash drop  
$ git stash apply  
$ git stash clear
```

Documentación [git stash](#)

Ignorar Archivos

.gitignore

Es un archivo de git que se crea en el repositorio en el cual se esta trabajando. Permite ignorar archivos o directorios de los que no deseamos hacer seguimiento.

Creación del archivo .gitignore

```
$ touch .gitignore
```

```
$ vim .gitignore
```

Documentación [.gitignore](#)

Resolución de conflictos

```
paoto@DESKTOP-0T7LS2T MINGW64 /d/Documentos/00-Research/projects/hello-git-fork (main)
$ git push
To https://github.com/paobtorres/hello-git.git
! [rejected]        main -> main (fetch first)
error: failed to push some refs to 'https://github.com/paobtorres/hello-git.git'
hint: Updates were rejected because the remote contains work that you do
hint: not have locally. This is usually caused by another repository pushing
hint: to the same ref. You may want to first integrate the remote changes
hint: (e.g., 'git pull ...') before pushing again.
hint: See the 'Note about fast-forwards' in 'git push --help' for details.
```

```
$ git diff
$ git reset <nombre_archivo>
$ git reflog
```

Merge conflict

```
If you have questions, please  
<<<<<< HEAD  
open an issue  
=====  
ask your question in IRC.  
>>>>>> branch-a
```

GitHub funcionalidades

- Markdown
- Issues
- Organizaciones
- GitHub Pages (version Paga)
- GitHub Co-Pilot (version Paga)

Markdown: readme.md

Es un archivo que debe incluir una descripción general del proyecto, instrucciones de instalación y configuración. Si es un repositorio publico es recomendable agregar información para constribuir. Ejemplo de archivo [README.md](#)

Ref: Documentación [markdown](#)


```
① README.md X
① README.md > ...
1 <picture>
2   <source media="(prefers-color-scheme: dark)" srcset="https://github.com/pyMBE-dev/pyMBE/blob/
   logos/logo_banner_dark_mode.png">
3   <source media="(prefers-color-scheme: light)" srcset="https://github.com/pyMBE-dev/pyMBE/blob/
   logos/logo_banner.png">
4   
5 </picture>
6
7 # pyMBE: the Python-based Molecule Builder for ESPResSo
8
9 ![GitHub Actions](https://github.com/pyMBE-dev/pyMBE/actions/workflows/testsuite.yml/badge.svg)
10 ![Codecov](https://codecov.io/gh/pyMBE-dev/pyMBE/branch/main/graph/badge.svg) | (https://codecov.
   io/gh/pyMBE-dev/pyMBE)
11 [[docs](https://badgen.net/static/docs/up-to-date/blue/)](pymbe-dev.github.io/pyMBE/
   html)
12 [[License: GPL v3](https://img.shields.io/badge/License-GPLv3-blue.svg)](LICENSE.txt)
13 [[Contributor Covenant](https://img.shields.io/badge/Contributor%20Covenant-2.1-4baaaa.svg)]
   (CODE_OF_CONDUCT.md)
14
15 [comment]: # (introduction start)
16
17 pyMBE provides tools to facilitate building up molecules with complex architectures in the
   Molecular Dynamics software [ESPResSo](https://espressomd.org/wordpress/). Some examples of
   molecules that can be set up with pyMBE are polyelectrolytes, hydrogels, peptides and globular
   proteins. pyMBE bookkeeps all the information about the molecule topology, permitting to link
   each particle to its corresponding residue and molecule. pyMBE uses the [Pint](https://pint.
   readthedocs.io/en/stable/) library to enable input parameters in any arbitrary unit system,
   which is later transformed in the reduced unit system used in ESPResSo.
18
19 [comment]: # (introduction stop)
20
21 An up-to-date documentation of all methods of the library can be found [here](pymbe-dev.github.
   io/pyMBE/pyMBE.html) and in the source code.
22
23 ## Dependencies
24
25 [ESPResSo](https://espressomd.org/wordpress/)
```

pyMBE-dev / pyMBE

Code Issues 10 Pull requests 2 Actions Projects Wiki Security and quality Insights Settings

is:issue state:open

Open 10 Closed 39

Author Labels Projects Milestones Assignees Types Newest

- Add support for nanoparticle models enhancement

Feature #144 · pm-blanco opened on Oct 21, 2025 · [pyMBEv2.0.0](#)

1 1
- Add support for all bonded potentials enhancement

Feature #143 · pm-blanco opened on Oct 21, 2025 · [pyMBEv2.0.0](#)

1
- Add support for multiprotic acids and bases enhancement

Feature #142 · pm-blanco opened on Oct 21, 2025 · [pyMBEv2.0.0](#)

1
- The functional test for hydrogels needs to be optimized ci-improvement

#122 · pm-blanco opened on May 9, 2025

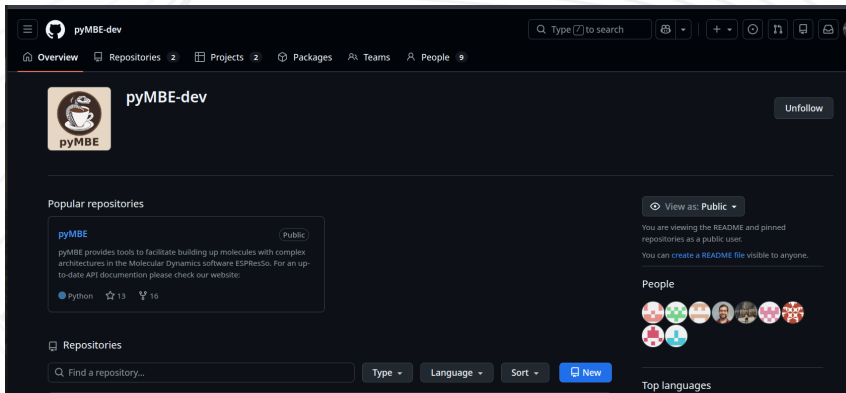
1
- Add support to create dendrimers enhancement

Feature #67 · pm-blanco opened on May 31, 2024 · [pyMBEv2.0.0](#)

3
- Add feature for citing resources in pyMBE documentation

#54 · pm-blanco opened on May 7, 2024

Organizaciones



Ref: Documentación [organizaciones](#)

Administración de repositorio

- Agregar usuarios
- Modificar Permisos
- Proteger Ramas
- Eliminar repositorio
- Renombrar



Herramientas gráficas



Desktop

GitHub desktop: [Pagina oficial](#)



GitKraken

GitKraken: [Pagina oficial](#)



Atlassian

SourceTree

SourceTree: [Pagina oficial](#)

GUI clients

Buenas practicas

- Nombrar los archivos/carpetas/repositorios sin espacios, ni letras como ñ o acentos.
- Realizar `commits` frecuentemente y con indicaciones claras de las modificaciones
- Utilizar ramas para desarrollo de tareas especifica
- Continuar aprendiendo y manejar ingles.

Documentación



API Documentation
for Python Projects

pdoc demo.py

```
demo.py

"""
A small 'pdoc' example.
"""

class Dog:
    """The name of our dog."""
    name: str
    friends: list["Dog"]
    """The friends of our dog."""

    def __init__(self, name: str):
        """Make a Dog without any friends (yet)."""
        self.name = name
        self.friends = []

    def bark(self, loud: bool = True):
        """WOOF"""
```

demo

A small `pdoc` example.

[View Source](#)

class Dog:

[View Source](#)



`Dog(name: str)`

[View Source](#)

Make a Dog without any friends (yet).

`name: str`

pdoc auto-generates API documentation that follows your project's Python module hierarchy. It requires no configuration, has first-class support for type annotations, cross-links between identifiers, comes with an integrated live-reloading web server, and understands numpydoc or Google-style docstrings.

Trabajo Final

Primera Parte: Grupal

- Crear un repositorio y agregar los integrantes
- Editar un script de python o un archivo txt en diferentes ramas.
- Cada integrante debera crear una rama con el siguiente nombre agregar-X-apellido.
- Una vez terminada su parte, deberan crear un pull-request de su rama y agregar en **Reviewers** paobtorres y Dany1970.

Segunda Parte: Individual

- Crear un repositorio y agregar: paobtorres y Dany1970.
- Agregar un archivo de texto con varias lineas o un codigo en python.
- Desafio: resolver un conflicto de merge.

1. Introducción

- ¿Qué es Git?
- ¿Cómo funciona Git?
- ¿Cómo se relaciona Git con GitHub u otros servicios?

2. Comandos básicos

- Comandos útiles de terminal (bash)
- Configuración básica de Git
- Creación de un repositorio Git
- Confirmar cambios en el repositorio
- Gestión de archivos
- Historial de cambios y estado del repositorio
- Hash: identificador de commits

3. Colaboración

- Sincronización
- Ramas
- Resolución de conflictos
- GitHub funcionalidades
- Buenas practicas

4. Referencias

Referencias

- Documentación oficial de [git](#)
- Documentación de [GitHub](#) (español)
- Atajos de git: [cheat-sheet](#)

Agradecimientos

- Facultad Regional San Rafael - UTN
- Soft Matter group from Charles University



CHARLES
UNIVERSITY

IN CASE OF FIRE



1. git commit



2. git push



3. git out!